

Функционалното програмиране е декларативен стил за програмиране, в който се издъзват страничните ефекти. Една функционална програма не описва как стъпка по стъпка да се правят нещата, а какво е резултатът. Например в Haskell можем да си направим сортираща функция така:

`sort [3] = [3]`

`sort (x:xs) = sort [y | y < x, y < xs] ++ [x] ++ sort [y | y <= x, y > xs]`

Ние само описваме какво е сортиран аргумент, а не казваме стъпка по стъпка как се сортира. Понесете издъзване страничните ефекти, вместо императивни конструкции като цикли ползваме рекурсия. Също така при едни-ви входни данни се връща едни и същи резултат, което позволява да запазим резултатите за да издъзнем повторно пресмятане (мемоизация), паралелизация на независими части от програмата. Функциите във функционалното програмиране са първокласни стойности и могат да се аргумент на друга функция, да дават резултат или да се свързват с имена

Основните компоненти са примитивни изрази, средства за кодиране и средства за абстракция:

- примитивните изрази са литерали и символи. Оценката на литерал е самият той, а на символ е стойността свързана с него. В схемата:

`5` се оценява на `5`

`#t` се оценява на `#t`

`+` се оценява на `procedure: +`

`(define a 7)`

`a` ще се оцени на `7`

- средства за кодиране - прилагане функция към аргументи. Например:

`(+ 1 2 3)`; оценява се до `6`

- средства за абстракция

- именуване (`define`)

- дефиниране на функция

```
(define pi 3.14) ; въвежда pi с 3.14
(define (square x) (* x x)) ; дефинира функция
(define (plus1 x) (+ x 1))
(define (compose f g) (lambda (x) (f (g x))))
((compose square plus1) 4)
```

$(\text{define } (f\ x) \dots) \Leftrightarrow (\text{define } f\ (\text{lambda } (x) \dots))$, т.е.:

$(\text{define } (\text{plus1 } x) (+ x 1)) \Leftrightarrow (\text{define } \text{plus1 } (\text{lambda } (x) (+ x 1)))$

Оценяване

Основното правило за оценяване е: оценяват се всички подизрази и след това функцията се прилага над тях.

$(\text{+ } (-11\ 1) (\text{+ } 2\ 3))$; $\Rightarrow (\text{+ } 10\ 5) \Rightarrow 50$

Изказание от това правило етс. Например `define`, `if`, `cond`, `lambda`, `and`, `or`, `'`

$(\text{define } x\ 7)$; не оценява x

$(\text{if } \#t\ 1\ (1\ 10))$; оценява само издирания блок - безгреша 1

$(\text{cond } (\#t\ 1))$; supra да оценява след първото $\#t$
 $(\#t\ (1\ 10))$

$(\text{lambda } (x) (+ 1\ x))$; не оценява при създаване

$(\text{and } \#f\ (1\ 10))$; supra при първото $\#f$

$(\text{or } \#t\ (1\ 10))$; supra при първото $\#t$

В scheme дефинираме функцията така:

$(\text{define } (\text{square } x) (* x x))$

$(\text{define } (\text{plus1 } x) (+ 1 x))$

$(\text{define } (f\ x\ y) (+ (\text{square } (\text{plus1 } x)) (\text{square } (\text{plus1 } y))))$

и $(f\ 1\ 2)$ се оценява така:

(f 1 2)

(+ (square (plus 1)) (square (plus 2)))

(+ (square (+ 1 1)) (square (+ 1 2)))

(+ (square 2) (square 3))

(+ (* 2 2) (* 3 3))

(+ 4 9)

15

Модел на оценяване

- Апликативно

Това е начинът по който scheme оценява и който използваме по-горе. Първо се оценяват аргументите и после се заместят.

(square (+ 1 2)) $\rightarrow$ (square 3) $\rightarrow$ (* 3 3) $\rightarrow$ 9

- Нормално

Аргументите се заместят неограничено и се преследват колкото по-далеч.

Хипотетично ако scheme оценяваше така (не оценява така)

(square (+ 1 2)) $\rightarrow$ (* (+ 1 2) (+ 1 2)) $\rightarrow$ (* 3 3) $\rightarrow$ 9

Haskeи оценява така

Възможно е апликативното оценяване да не завърши, а ленивото да завърши

Например

Haskeи

Scheme

first x y = x

(define (first x y) x)

first 7 (div 1 0)

(first 7 (1 1 0))

$\uparrow$

брой 7, а

(div 1 0) не го
оценява

$\uparrow$

гласи, защото
се продава да
оцени (1 1 0)

Друг подобен пример е:

Haskell

first x y = x

loop = loop

first 7 loop

↑
броя 7, и loop
възвръща не го
оценява

Scheme

(define (first x y) x)

(define (loop) (loop))

(first 7 (loop))

↑
зацикля, защото
се провага
оценя (loop)

От теорема за нормализацията знаем, че ако някоя стратегия на оценяване стигне до резултат, то и нормалната стратегия стигне до резултат и то същия. От тук следва, че ако нормалната стратегия гадже зрешка/не завърши, то никаква друга няма да успее.

Функция от по-висок ред е функция, която приема функция като параметър и/или връща функция. Например:

```
(define (compose f g) (lambda (x) (f (g x))))
```

Взимам две функции и връща функция

```
(map square '(1 2 3))
```

Взимам функция и лист и връща лист. В случая (1 4 9)

```
(define (plus1 x) (+ 1 x))
```

```
((compose square plus1) 4)
```

```
(lambda (x) (square (plus1 x) 4))
```

```
(square (plus1 4))
```

```
(square (+ 1 4))
```

```
(square 5)
```

```
(* 5 5)
```

25

В Haskell функциите на няколко аргумента се разглеждат като верига от функции на един аргумент, т.е. всяка приема аргумент и връща следващата функция от веригата, т.е.

$$f :: a \rightarrow b \rightarrow c \quad \Leftrightarrow \quad f :: a \rightarrow (b \rightarrow c)$$

Когато някоя функция ни трябва веднъж е удобно да се използва lambda

Пример:

```
(map (lambda (x) (* 5 x)) '(1 2 3)); (5 10 15)
```

Така при тази функция не е нужно да и даваме име, което е удобно за функции от по-висока ред.

Ето малка брета функция която гофичираме с измислен lambda:

Haskell

```
(define (plus1 x) (+ 1 x))  $\Leftrightarrow$  (define plus1 (lambda (x) (+ 1 x)))
```

Scheme

```
plus1 x = x + 1  $\Leftrightarrow$  plus1 =  $\lambda x \rightarrow x + 1$ 
```